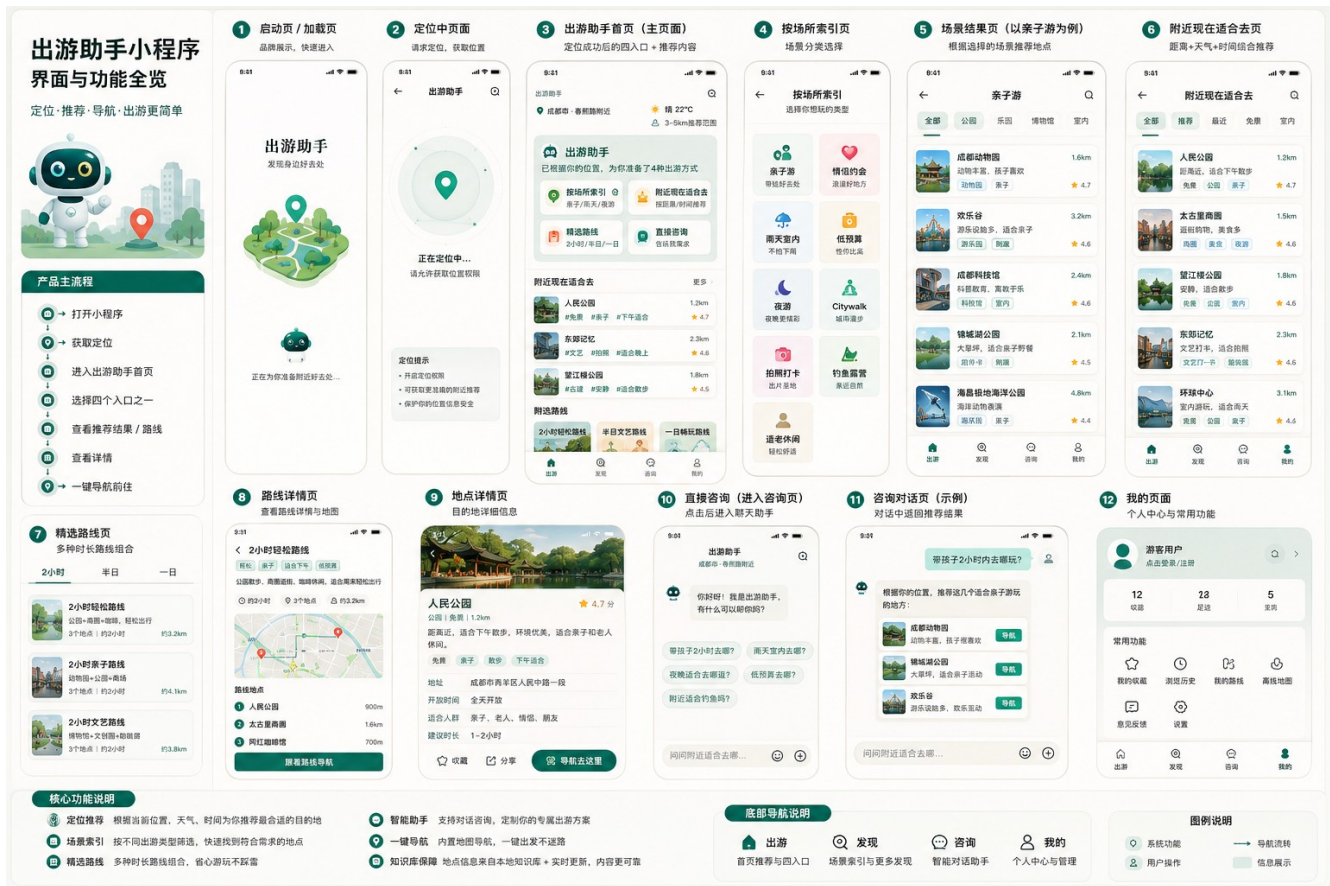


出游助手小程序

travel_v2 重构完整方案

定位 · 推荐 · 场景 · 路线 · 咨询 · 知识库闭环



文档版本	V1.0
适用项目	https://github.com/Zhangwei930/travel_v2
目标方向	打开小程序 - 定位 - 出游助手 - 四个入口 - 地点/路线/咨询 - 一键导航
生成日期	2026-05-24
核心结论	不建议整合推倒重来。保留现有 UniApp/Vue3、FastAPI、SQLAlchemy、地图/AI/WebSearch 可插拔基础，但必须重做前端主流程、底部导航、首页四入口、推荐引擎、数据库场景化结构和知识库闭环。

目录

1. 项目重新定位与跑偏判断
2. 产品主流程与信息架构
3. UI 页面重构方案
4. 前端架构与文件改造清单
5. 后端架构与服务拆分
6. API 接口设计
7. 数据库设计
8. 知识库与联网搜索闭环
9. 推荐引擎与路线引擎
10. 开发落地步骤与迁移计划
11. 测试验收标准
12. 给 Codex/Claude Code 的执行提示词
13. 附录：关键配置、DDL、接口示例

1. 项目重新定位与跑偏判断

1.1 新产品定位

本小程序的核心不是“远程旅游攻略生成”，而是“定位驱动的附近出游助手”。用户打开小程序后，系统先获取当前位置，再根据距离、天气、时间、场景、热度和知识库，告诉用户现在附近最适合去哪、为什么适合、怎么过去。

一句话定位
打开小程序，自动定位，直接告诉我附近现在适合去哪，并能按场景索引、精选路线、智能咨询、一键导航。

1.2 当前仓库与原型图的核心差异

当前仓库仍保留“周密出游 - 智能本地出游规划系统”的产品表达，前端底部 Tab 为：首页、场景、生成、助手、我的。后端也已有攻略生成、知识库问答、路线推荐、反馈等基础接口。新的原型图要求主流程变成：出游、发现、咨询、我的四个底部入口，并且首页强调定位后的四个入口：按场所索引、附近现在适合去、精选路线、直接问。

维度	当前偏向	原型图目标	处理建议
底部导航	首页 / 场景 / 生成 / 助手 / 我的	出游 / 发现 / 咨询 / 我的	必须改，去掉主 Tab 的“生成”
首页价值	展示攻略生成、场景、路线	定位后给附近适合去哪	首页重做为定位推荐中枢
核心按钮	一键生成攻略	四入口：索引、附近、路线、咨询	生成能力降级为咨询内部能力
数据核心	攻略结果与 FAQ	POI 知识、场景规则、路线组合、反馈闭环	数据库补强，做知识沉淀
用户决策	看攻略文本	看地点卡片、路线、详情、导航	详情页和导航按钮优先

1.3 改造原则

- 不整仓重建：保留可复用工程底座，减少风险。
- 主流程重做：UI、Tab、首页、页面路径按原型图重新组织。
- 推荐优先：从“用户填表生成攻略”改为“系统主动推荐附近选择”。
- 知识库闭环：有内容优先返回库；不完整则联网补充；新增内容先入待审核。
- 一键导航闭环：每个地点、每条路线都要能跳转地图导航。

2. 产品主流程与信息架构

2.1 主流程

- 打开小程序
- > 启动页/加载页
 - > 请求定位授权
 - > 定位成功，获得 lat/lng/city/district
 - > 拉取天气、时间段、附近 POI、场景规则、路线模板
 - > 进入“出游”首页
 - > 四个入口：按场所索引 / 附近现在适合去 / 精选路线 / 直接问
 - > 查看场景结果、地点详情、路线详情或咨询结果
 - > 一键导航、收藏、反馈
 - > 反馈与联网补充进入知识库待审核

2.2 四个底部 Tab

Tab	页面路径	职责	首页优先级
出游	pages/travel/index	定位后的主入口，展示四入口、附近推荐、精选路线	最高
发现	pages/discover/index	场景索引、热门场所、专题合集	高
咨询	pages/consult/chat	带定位、天气、知识库和地图能力的出游助手	高
我的	pages/profile/index	收藏、历史、我的路线、反馈、设置	中

2.3 产品信息架构

出游
- 当前定位 / 天气 / 推荐半径
- 四入口
- 按场所索引
- 附近现在适合去
- 精选路线
- 直接问出游助手
- 附近现在适合去列表
- 精选路线列表
发现
- 亲子游 / 情侣约会 / 雨天室内 / 低预算 / 夜游 / Citywalk / 拍照打卡 / 钓鱼露营 / 适老休闲
- 场景结果页
咨询
- 快捷问题
- 聊天回答
- POI 卡片 / 路线卡片 / 来源说明 / 导航按钮
我的
- 我的收藏
- 浏览历史
- 我的路线
- 意见反馈
- 设置

3. UI 页面重构方案

3.1 设计语言

项目	建议
主色	#0D4F4A / #00796B，贴合原型绿色系
背景	#F5F1EB 或 #F7F8F5，卡片白色
卡片	圆角 16-24，轻阴影，信息不拥挤
图标	线性图标 + 绿色实心强调，地图定位图标突出
列表密度	移动端一屏展示 3-4 张地点卡片，避免攻略长文

按钮	主按钮为“导航去这里”“跟着路线导航”，次按钮为收藏/分享
----	-------------------------------

3.2 启动页/加载页

- 展示产品名“出游助手”和 Slogan “发现身边好去处”。
- 进行初始化：读取本地定位缓存、检查授权状态、准备首页请求。
- 不要停留太久，1 秒内进入定位页或首页。

3.3 定位页

- 核心文案：正在定位中，请允许获取位置权限。
- 说明定位用途：推荐附近地点、判断天气和时间、生成路线。
- 失败状态提供：重新定位、手动选择城市、继续使用默认城市。

3.4 出游首页

区域	内容	接口来源
顶部栏	城市/区域、搜索按钮	/api/home/bootstrap
天气/范围	晴 22°C、3-5km、适合户外	/api/weather/current
主卡片	出游助手：已根据你的位置准备 4 种方式	/api/home/bootstrap
四入口	按场所索引、附近现在适合去、精选路线、直接问	前端固定 + 后端配置
附近推荐	3-5 个 POI 卡片	/api/nearby/recommend
精选路线	2 小时、半日、一日路线	/api/routes/featured

3.5 按场所索引页

场景	适合类型	排除类型	关键字段
亲子游	公园、博物馆、动物园、科技馆	危险山路、酒吧、夜店	安全、厕所、停车、休息区
情侣约会	街区、夜景、展馆、咖啡、商圈	人流过大、无氛围、太吵	氛围、拍照、餐饮、夜景
雨天室内	博物馆、商场、书店、室内乐园	露天公园、山地、水边	室内、交通、预约、拥挤度
低预算	免费公园、街区、展馆、城市步道	高门票、高消费场所	免费/低价、交通成本
夜游	夜市、灯光街区、商圈、江边	偏僻区域、闭园地点	营业时间、安全、灯光
Citywalk	老街、河边、文化街区、商圈串联	远距离跳点、交通断点	步行强度、补给点、路线
拍照打卡	地标、展馆、街区、自然景观	普通商铺、无特色点位	光线、机位、人流
钓鱼露营	湖边、营地、郊野公园	禁钓水域、危险区域	天气、风力、停车、装备
适老休闲	公园、茶馆、平缓步道、展馆	台阶多、强度大、拥挤	座椅、厕所、无障碍

3.6 场景结果页

- 顶部显示当前场景：例如“亲子游”。
- 筛选项：全部、公园、乐园、博物馆、室内。
- 卡片字段：封面、名称、距离、标签、评分、推荐理由、导航按钮。
- 页面底部保留 Tab，不跳出主流程。

3.7 附近现在适合去页

- 筛选项：全部、推荐、最近、免费、室内。
- 默认使用综合推荐排序，不简单按距离排序。
- 列表展示距离、天气适配、时间适配、场景标签、评分。

- 每张卡片必须有“导航”入口。

3.8 精选路线页与路线详情页

- 路线列表分为：2 小时、半日、一日。
- 路线详情包含地图预览、路线节点、建议停留时间、每站推荐理由、注意事项。
- 路线不是攻略长文，而是可执行路径。
- 底部主按钮：跟着路线导航。

3.9 地点详情页

模块	必须展示
基础信息	封面、名称、评分、距离、地址、开放时间
适合判断	现在是否适合去、适合人群、适合天气、建议时长
推荐理由	为什么推荐，不能只显示地图简介
避坑提醒	预约、闭馆、人流、停车、雨天风险
路线搭配	附近还可以一起去哪里
底部操作	收藏、分享、导航去这里

3.10 咨询页

- 快捷问题：附近 2 小时去哪？今天雨天适合去哪？带孩子去哪？晚上去哪？低预算去哪？
- 回答必须带 POI/路线卡片，不能只输出文字。
- 回答必须显示来源：地图、天气、知识库、联网搜索。
- 回答必须可操作：导航、收藏、查看详情、换一批。

4. 前端架构与文件改造清单

4.1 新目录结构

```
src/  
  pages/  
    launch/index.vue  
    location/index.vue  
    travel/index.vue  
    discover/index.vue  
    discover/scene-result.vue  
    nearby/index.vue  
    route/list.vue  
    route/detail.vue  
    poi/detail.vue  
    consult/chat.vue  
    profile/index.vue  
    profile/favorites.vue  
    profile/history.vue  
    profile/routes.vue  
    profile/feedback.vue  
    profile/settings.vue
```

```
components/  
  AppHeader.vue  
  LocationBar.vue  
  WeatherBadge.vue  
  EntryCard.vue  
  SceneGrid.vue  
  PoiCard.vue  
  RouteCard.vue  
  RouteTimeline.vue  
  ChatBubble.vue  
  SourceBadge.vue  
  EmptyState.vue  
  BottomTabBar.vue
```

```
store/  
  location.js  
  home.js  
  scene.js  
  nearby.js  
  route.js  
  consult.js  
  user.js
```

```
api/  
  request.js  
  travel.js  
  location.js  
  nearby.js  
  route.js  
  consult.js  
  user.js
```

4.2 pages.json 改造目标

```
{  
  "pages": [  
    { "path": "pages/launch/index", "style": { "navigationStyle": "custom", "navigationBarTitleText": "出游助手" } },  
    { "path": "pages/location/index", "style": { "navigationStyle": "custom", "navigationBarTitleText": "定位" } },  
    { "path": "pages/travel/index", "style": { "navigationStyle": "custom", "navigationBarTitleText": "出游" } },  
    { "path": "pages/discover/index", "style": { "navigationStyle": "custom", "navigationBarTitleText": "发现" } },  
    { "path": "pages/consult/chat", "style": { "navigationStyle": "custom", "navigationBarTitleText": "咨询" } },  
    { "path": "pages/profile/index", "style": { "navigationStyle": "custom", "navigationBarTitleText": "我的" } }  
  ],  
  "tabBar": {  
    "custom": true,  
    "color": "#7A8B86",
```

```
"selectedColor": "#00796B",
"backgroundColor": "#FFFFFF",
"list": [
  { "pagePath": "pages/travel/index", "text": "出游" },
  { "pagePath": "pages/discover/index", "text": "发现" },
  { "pagePath": "pages/consult/chat", "text": "咨询" },
  { "pagePath": "pages/profile/index", "text": "我的" }
]
}
```

4.3 现有页面处理策略

当前页面	处理方式	新归属
pages/index/index	重构或迁移为 travel/index	出游首页
pages/scenes/scenes	迁移为 discover/index	发现/场景索引
pages/generate/generate	不要作为 Tab，降级为咨询内部的计划表单	consult/plan-form
pages/assistant/chat	迁移为 consult/chat	咨询页
pages/profile/profile	迁移为 profile/index	我的
pages/result/result	保留为内部结果页或并入路线详情	route/detail 或 consult/result
pages/admin/admin	不放入小程序主流程，作为后台入口	admin 独立

4.4 前端状态 Store

Store	状态字段	职责
location	lat/lng/city/district/address/permission	定位缓存、城市切换、授权状态
home	weather/entries/nearby/routes/loading	首页初始化数据
scene	sceneList/currentScene/resultPois/resultRoutes	场景索引与结果
nearby	filters/list/radius/sort	附近推荐列表
route	featured/detail/generateParams	路线列表与详情
consult	sessionId/messages/quickQuestions	聊天会话和回答卡片
user	openid/profile/favorites/history	用户信息、收藏、历史

4.5 前端 API 封装

```
// src/api/travel.js
import { request } from './request.js'

export const travelApi = {
  bootstrap: (params) => request('/api/home/bootstrap', { data: params }),
  getScenes: () => request('/api/scenes'),
  getScenePois: (sceneKey, params) => request(`/api/scenes/${sceneKey}/pois`, { data: params }),
  getNearby: (params) => request('/api/nearby/recommend', { data: params }),
  getPoiDetail: (params) => request('/api/poi/detail', { data: params }),
  getFeaturedRoutes: (params) => request('/api/routes/featured', { data: params }),
  getRouteDetail: (params) => request('/api/routes/detail', { data: params }),
  ask: (payload) => request('/api/consult/ask', { method: 'POST', data: payload }),
}
```


5. 后端架构与服务拆分

5.1 后端总架构

FastAPI API 层

- location_router: 定位反查、城市选择
- weather_router: 当前天气、天气建议
- home_router: 首页 bootstrap 聚合接口
- scene_router: 场景索引、场景结果
- nearby_router: 附近现在适合去
- poi_router: 地点详情、导航参数、附近搭配
- route_router: 精选路线、路线详情、动态路线生成
- consult_router: 出游助手问答
- kb_router: 知识库检索、待审核
- user_router: 收藏、历史、个人设置
- feedback_router: 反馈、纠错、评分

服务层

- map_provider: 高德/腾讯地图适配
- weather_provider: 天气适配
- recommend_engine: 推荐评分
- scene_engine: 场景规则匹配
- route_engine: 路线组合
- kb_retriever: 结构化库 + 向量库检索
- web_search_service: 联网搜索与缓存
- llm_service: Dify/Ollama/GPT/Claude 等适配
- review_service: 待审核知识流转

5.2 后端目录建议

```
server/app/  
main.py  
config.py  
database.py
```

```
routers/  
home.py  
location.py  
weather.py  
scene.py  
nearby.py  
poi.py  
route.py  
consult.py  
kb.py  
user.py  
feedback.py
```

admin.py

services/

location_service.py

map_provider.py

weather_provider.py

recommend_engine.py

scene_engine.py

route_engine.py

consult_service.py

kb_retriever.py

web_search_service.py

llm_service.py

review_service.py

models/

user.py

poi.py

scene.py

route.py

kb.py

feedback.py

chat.py

schemas/

home.py

poi.py

route.py

consult.py

kb.py

5.3 首页聚合接口逻辑

GET /api/home/bootstrap

输入: lat/lng/city/radius

处理:

1. 如果 city 为空, 根据 lat/lng 调 location_service 反查城市、行政区、商圈。
2. 调 weather_provider 获取当前天气。
3. 根据当前时间生成 time_slot: morning/afternoon/night。
4. 调 map_provider 搜索附近 POI, 优先使用缓存, 过期后刷新。
5. 调 recommend_engine 对 POI 打分排序。
6. 调 route_engine 获取 2 小时/半日/一日精选路线。
7. 返回首页需要的完整数据。

5.4 旧接口兼容策略

旧接口	新接口	兼容建议
/api/weather	/api/weather/current	保留旧接口 1 个版本，内部转发到新服务
/api/scene/list	/api/scenes	保留 alias，前端逐步迁移
/api/poi/list	/api/nearby/recommend 或 /api/scenes/{scene}/pois	按参数分流
/api/route/recommend	/api/routes/featured	保留兼容，但响应字段统一
/api/trip/generate	/api/routes/generate 或 /api/consult/ask	降级为内部能力
/api/kb/ask	/api/consult/ask	保留旧入口，统一走 consult_service

6. API 接口设计

6.1 接口总表

模块	接口	方法	说明
首页	/api/home/bootstrap	GET	首页一次性初始化
定位	/api/location/reverse	GET	经纬度反查城市/区域/商圈
天气	/api/weather/current	GET	当前天气与出游建议
场景	/api/scenes	GET	获取 9 个场景
场景	/api/scenes/{scene_key}/pois	GET	场景推荐地点
场景	/api/scenes/{scene_key}/routes	GET	场景推荐路线
附近	/api/nearby/recommend	GET	附近现在适合去
地点	/api/poi/detail	GET	地点详情
地点	/api/poi/nav	GET	返回地图导航参数
路线	/api/routes/featured	GET	精选路线
路线	/api/routes/detail	GET	路线详情
路线	/api/routes/generate	POST	按位置与偏好动态生成路线
咨询	/api/consult/ask	POST	出游助手问答
用户	/api/user/favorite	POST	收藏/取消收藏
用户	/api/user/favorites	GET	我的收藏
用户	/api/user/history	GET	浏览历史
反馈	/api/feedback	POST	用户反馈/纠错
管理	/api/admin/kb/pending	GET	待审核知识列表
管理	/api/admin/kb/approve	POST	审核入库

6.2 /api/home/bootstrap 示例

```
GET /api/home/bootstrap?lat=30.67&lng=104.06&radius=5000

Response:
{
  "location": {
    "city": "成都市",
    "district": "青羊区",
    "label": "青羊附近",
    "lat": 30.67,
    "lng": 104.06
  },
  "weather": {
```

出游助手小程序 travel_v2 重构完整方案

```
"temp": 22,
"condition": "晴",
"time_slot": "afternoon",
"advice": "适合户外散步和亲子出行"
},
"entries": [
  { "key": "scene", "title": "按场所索引" },
  { "key": "nearby", "title": "附近现在适合去" },
  { "key": "route", "title": "精选路线" },
  { "key": "consult", "title": "直接问" }
],
"nearby_recommend": [],
"featured_routes": []
}
```

6.3 /api/nearby/recommend 参数

参数	类型	说明
lat/lng	float	当前位置，经纬度
city	string	城市，可由后端反查
radius	int	推荐半径，默认 5000 米
filter	enum	all/recommend/nearest/free/indoor
scene	string	family/couple/rainy/budget/night/citywalk/photo/camping/elder
time_slot	enum	morning/afternoon/night，可后端自动判断
weather	string	sunny/rainy/snow/windy/hot/cold，可后端自动获取

6.4 /api/consult/ask 示例

```
POST /api/consult/ask
{
  "question": "附近 2 小时去哪玩？ ",
  "lat": 30.67,
  "lng": 104.06,
  "city": "成都",
  "weather": "晴",
  "history": []
}
```

Response:

```
{
  "answer": "根据你当前位置、天气和知识库，推荐以下地点。",
  "cards": [
    {
      "type": "poi",
      "id": 1001,
      "name": "人民公园",
      "distance": "1.2km",
```

```
    "reason": "距离近，适合下午散步，亲子和适老都友好。",
    "tags": ["免费", "亲子", "散步"],
    "action": "nav"
  }
],
"sources": [
  { "type": "map", "name": "地图 POI" },
  { "type": "weather", "name": "实时天气" },
  { "type": "kb", "name": "本地知识库" }
]
```

7. 数据库设计

7.1 数据库分层

层级	用途	表
基础数据层	用户、定位、POI 基础索引	users, user_profile, poi_cache
场景规则层	场景分类、规则、POI 关系	scenes, scene_rules, poi_scene_rel
知识增强层	推荐理由、避坑、适配信息	poi_knowledge, kb_documents, kb_chunks
路线层	路线模板和路线节点	routes, route_stops
交互层	收藏、历史、聊天、反馈	favorites, visit_history, chat_sessions, chat_messages, user_feedback
审核层	联网/AI 生成内容待审核	kb_pending, review_tasks

7.2 核心表结构

```
users
- id
- openid
- nickname
- avatar_url
- city
- created_at
- last_login_at

user_profile
- id
- user_id
- preferred_scenes
- preferred_budget
- preferred_transport
- family_type
- created_at
- updated_at

scenes
- id
```

- scene_key
- name
- icon
- color
- description
- sort_no
- enabled

scene_rules

- id
- scene_key
- suitable_categories
- exclude_categories
- suitable_weather
- suitable_time_slots
- suitable_people
- tags
- rule_json

poi_cache

- id
- provider
- provider_poi_id
- name
- city
- district
- address
- lat
- lng
- category
- tel
- rating
- cost_level
- is_free
- is_indoor
- cover_image
- fetched_at
- expires_at
- created_at
- updated_at

poi_knowledge

- id
- poi_id
- recommend_reason
- suitable_people

- suitable_weather
- suitable_time
- best_time
- play_duration
- budget_desc
- avoid_tips
- parking_info
- toilet_info
- child_friendly
- elder_friendly
- source_type
- review_status
- updated_at

poi_scene_rel

- id
- poi_id
- scene_key
- score
- reason
- created_at

routes

- id
- route_no
- title
- city
- scene_key
- duration_type
- total_distance
- total_time
- budget_desc
- suitable_people
- cover_image
- summary
- tips
- source_type
- review_status
- created_at
- updated_at

route_stops

- id
- route_id
- poi_id
- stop_order

- arrive_time
- stay_duration
- transport
- reason
- tip

7.3 生产数据库建议

- 本地开发可继续使用 SQLite，方便零配置运行。
- 生产建议 PostgreSQL + pgvector，用于结构化数据和向量检索。
- Redis 用于接口缓存、定位结果缓存、搜索缓存、异步任务队列。
- 地图 API 返回的完整数据不要永久全量保存，只存必要索引字段和缓存过期时间。

8. 知识库与联网搜索闭环

8.1 知识库不是“攻略大全”

本项目的知识库应成为“本地出游判断能力”，而不是堆积长攻略。核心沉淀的是场景规则、地点推荐理由、避坑提醒、适合人群、适合天气、路线模板、用户反馈和联网更新结果。

8.2 四层知识体系

层级	说明	例子
结构化数据库	强约束、可排序、可筛选	POI、场景、路线、评分、标签
向量知识库	语义检索、回答解释	攻略说明、避坑、FAQ、场景规则解释
联网搜索缓存	补充不完整或实时信息	闭馆、临时活动、预约公告、天气影响
人工审核库	所有新增内容先 pending	AI/WebSearch 内容待审核后入库

8.3 更新流程

- 用户提问 / 首页推荐
- > 解析位置、时间、天气、场景
 - > 先查结构化库：POI、路线、场景规则
 - > 再查向量库：推荐理由、避坑提醒、FAQ
 - > 命中率足够：直接返回知识库内容
 - > 命中率不足：调用地图 API / WebSearch
 - > 生成临时答案，并标注来源和更新时间
 - > 写入 kb_pending 待审核
 - > 管理员审核后进入正式知识库

8.4 知识更新策略

情况	处理方式
库里有内容且未过期	直接返回知识库，不调用 WebSearch
库里有内容但不完整	返回已有内容，同时联网补充，补充进入 kb_pending
库里没有内容	调用地图 + WebSearch 生成临时答案，标注需核实，写入待审核
用户反馈纠错	进入 user_feedback，生成审核任务，审核后更新 poi_knowledge 或 kb_documents

8.5 风险控制

- 涉及门票、营业时间、闭馆、危险水域、禁钓等内容必须显示更新时间。
- 高风险内容不能自动入库，必须人工审核。
- AI 回答不能编造实时数据；没有确认来源时要提示“出发前请核实”。
- 用户反馈不能直接覆盖正式知识，必须经过审核。

9. 推荐引擎与路线引擎

9.1 附近推荐评分公式

```
score = 0
score += distance_score * 0.25
score += weather_score * 0.20
score += time_score * 0.20
score += scene_score * 0.15
score += popularity_score * 0.10
score += user_preference_score * 0.10

if poi_closed:
    score -= 50
if weather_risk:
    score -= 30
if not suitable_for_current_people:
    score -= 20
```

9.2 适配规则示例

条件	加分类型	降权类型
晴天下午	公园、湖边、Citywalk、拍照打卡	纯室内且距离远
雨天	博物馆、商场、书店、室内乐园	露天公园、山地、水边
晚上	夜市、灯光街区、商圈、夜景点	闭园地点、偏僻区域
带孩子	安全、厕所、停车、低强度、亲子设施	危险、拥挤、夜间、步行强度大
适老	平缓、座椅、厕所、休息点、交通方便	台阶多、坡道多、排队久
低预算	免费公园、公共街区、低价展馆	高门票、高消费商圈

9.3 路线组合规则

- 2 小时路线：站点 2-3 个，总步行距离优先控制在 2-4km。
- 半日路线：站点 3-5 个，允许一次短交通换乘。
- 一日路线：站点 4-6 个，必须安排午餐/休息节点。
- 路线节点之间不能过远，优先同区域串联。
- 路线必须考虑当前时间：下午不能推荐已经闭馆的上午路线。
- 路线必须给每个节点建议停留时间和下一站交通方式。

9.4 推荐引擎伪代码

```
def recommend_nearby(lat, lng, city, radius, scene=None):
```

```
location = location_service.resolve(lat, lng, city)
weather = weather_provider.current(location.city)
time_slot = get_current_time_slot()
pois = map_provider.search_nearby(lat, lng, radius)
knowledge = kb_retriever.load_poi_knowledge(pois)

results = []
for poi in pois:
    score_detail = recommend_engine.score(
        poi=poi,
        knowledge=knowledge.get(poi.id),
        scene=scene,
        weather=weather,
        time_slot=time_slot,
        user_profile=current_user_profile()
    )
    if score_detail.score > 0:
        results.append(build_poi_card(poi, score_detail))

return sorted(results, key=lambda x: x['score'], reverse=True)
```

10. 开发落地步骤与迁移计划

10.1 总体策略

推荐策略
开新分支进行“主流程重构”，不要在 main 上直接大改。先保留旧接口和旧页面，再逐步把新 UI 接到新接口，最后删除废弃入口。

10.2 阶段拆分

阶段	目标	交付物	建议耗时
第 0 阶段	保护现状、建立回滚点	分支、tag、savepoint patch	0.5 天
第 1 阶段	前端主导航重构	4 Tab、travel 首页、discover/consult/profile	1-2 天
第 2 阶段	首页 bootstrap 接口	home_router、location/weather 聚合	1 天
第 3 阶段	附近推荐和场景结果	nearby_router、recommend_engine、场景规则	2-3 天
第 4 阶段	路线与详情页	route_engine、route_stops、地图导航参数	2 天
第 5 阶段	咨询与知识库闭环	consult_service、kb_retriever、web_search、pending	3-5 天
第 6 阶段	测试、验收、上线	端到端测试、性能优化、埋点	2 天

10.3 Git 操作建议

```
# 保护当前进度
git status
git add .
```

```
git commit -m "chore: save current travel_v2 state before travel assistant refactor"
git tag before-travel-assistant-refactor
```

```
# 新建重构分支
git checkout -b refactor/travel-assistant-ui-architecture

# 每完成一个阶段提交一次
git add .
git commit -m "feat: redesign tabbar and travel home entry"
git commit -m "feat: add home bootstrap api"
git commit -m "feat: add nearby recommendation engine"
```

10.4 第一批必须改的文件

优先级	文件/目录	动作
P0	src/pages.json	改为 4 Tab，新增新页面路径
P0	src/pages/travel/index.vue	重做出游首页
P0	src/pages/discover/index.vue	九宫格场景索引
P0	src/pages/consult/chat.vue	咨询页替代旧助手页
P0	src/components/BottomTabBar.vue	底部导航改为 4 项
P1	server/app/routers/home.py	新增 bootstrap 聚合接口
P1	server/app/services/recommend_engine.py	新增推荐评分
P1	server/app/routers/nearby.py	新增附近推荐接口
P2	server/app/models.py 或 models/*	补充场景、POI 知识、路线节点、聊天表
P2	server/app/services/kb_retriever.py	重构知识库检索

10.5 不要一开始就做的事

- 不要一开始就接很多外部平台内容，否则会陷入数据源问题。
- 不要把“生成攻略”继续放在首页核心位置。
- 不要先做复杂后台管理，先让用户端主流程跑通。
- 不要让 AI 直接写正式知识库，必须先写入 kb_pending。
- 不要只按距离排序，否则无法体现“现在适合去”。

11. 测试验收标准

11.1 用流程收

流程	验收标准
首次打开	能看到启动页，能请求定位，定位失败有兜底
首页	能显示城市、天气、四入口、附近推荐、精选路线
按场所索引	9 个场景可点击，进入结果页后列表正确
附近推荐	筛选项可用，排序符合天气/时间/距离逻辑
地点详情	能看到推荐理由、避坑、适合人群、导航按钮
路线详情	能看到节点、停留时间、交通方式、路线导航
咨询	能根据定位回答，并返回卡片和来源
我的	收藏、历史、反馈入口可用

11.2 接口验收

- /api/home/bootstrap 在 2 秒内返回首页首屏必要数据。
- /api/nearby/recommend 支持 radius/filter/scene 参数。
- /api/consult/ask 必须返回 answer、cards、sources 三类字段。
- 所有接口失败时返回统一错误格式，前端能展示空状态。
- 旧接口至少保留一个过渡版本，避免前端迁移中断。

11.3 数据验收

- 每个 POI 至少具备 name、lat、lng、city、category、distance、tags。
- 每个知识增强 POI 至少有 recommend_reason、suitable_weather、suitable_people、avoid_tips。
- 每条路线至少有 2 个 route_stops。
- AI/WebSearch 新增内容默认进入 kb_pending，不直接进入正式库。
- 用户反馈能关联 target_type 和 target_id。

12. 给 Codex/Claude Code 的执行提示词

12.1 总提示词

你正在改造 Zhangwei930/travel_v2 项目。目标不是继续做“周密出游攻略生成器”，而是按照新的原型图重构为“定位驱动的附近出游助手小程序”。

核心要求：

1. 底部 Tab 改为：出游、发现、咨询、我的。
2. 首页路径改为 pages/travel/index，首页必须展示定位、天气、四入口、附近推荐、精选路线。
3. 四入口为：按场所索引、附近现在适合去、精选路线、直接问出游助手。
4. “生成攻略”不再作为主 Tab，只能作为咨询或路线内部能力。
5. 后端新增 /api/home/bootstrap、/api/nearby/recommend、/api/consult/ask。
6. 推荐逻辑必须综合距离、天气、时间、场景、热度、用户偏好，不能只按距离排序。
7. 知识库流程必须是：先查库；不完整再联网补充；AI/WebSearch 新增内容进入 kb_pending 待审核。
8. 尽量保留现有可运行结构，分阶段提交，不要一次性删除大量文件。

请先阅读 src/pages.json、src/api/mock.js、server/README.md、server/app/models.py，然后按阶段输出改造计划，再开始改代码。

12.2 第一阶段提示词：改 UI 主导航

请先完成前端主导航和页面骨架改造：

- 修改 src/pages.json，把 tabBar 改为 4 项：出游、发现、咨询、我的。
 - 新增或迁移页面：pages/travel/index、pages/discover/index、pages/consult/chat、pages/profile/index。
 - 原 pages/generate/generate 不再作为 tabBar 页面。
 - 出游首页先用静态 mock 展示：定位栏、天气、四入口、附近现在适合去、精选路线。
 - 保证 H5 和 mp-weixin 构建不报错。
- 完成后输出修改文件列表和运行命令。

12.3 第二阶段提示词：后端 bootstrap

请新增后端 /api/home/bootstrap 接口：

- 新建 server/app/routers/home.py。
- 在 app/main.py 注册 home router。
- 接口接收 lat/lng/city/radius。
- 返回 location、weather、entries、nearby_recommend、featured_routes。
- 初期可以复用现有 weather、poi、route 服务或 stub 数据。
- 返回字段必须与前端 travel/index.vue 对齐。
- 加入基础单元测试。

12.4 第三阶段提示词：推荐引擎

请新增推荐引擎：

- 新建 server/app/services/recommend_engine.py。
- 实现 distance_score、weather_score、time_score、scene_score、popularity_score、user_preference_score。
- 新增 /api/nearby/recommend 接口。
- 支持 filter=all/recommend/nearest/free/indoor。
- 返回每个 POI 的 score_detail，方便前端展示推荐理由。
- 不要只按距离排序。

13. 附录：关键配置、DDL、接口示例

13.1 .env 建议

```
DATABASE_URL=sqlite:///./travel.db
# 生产: postgresql+psycopg://user:password@127.0.0.1:5432/travel_assistant
```

```
MAP_PROVIDER=amap
AMAP_KEY=填写高德 Key
```

```
WEATHER_PROVIDER=amap
```

```
AI_PROVIDER=dify
DIFY_API_BASE=https://your-dify.example.com/v1
DIFY_API_KEY=填写 Dify Key
```

```
WEB_SEARCH_PROVIDER=searxng
SEARXNG_BASE=http://127.0.0.1:8080
```

```
ADMIN_TOKEN=请设置强随机字符串
KB_AUTO_APPROVE=false
CACHE_TTL_POI_HOURS=24
CACHE_TTL_SEARCH_HOURS=6
```

13.2 PostgreSQL DDL 摘要

```
CREATE TABLE scenes (
  id SERIAL PRIMARY KEY,
  scene_key VARCHAR(40) UNIQUE NOT NULL,
  name VARCHAR(80) NOT NULL,
```

```
icon VARCHAR(80),
color VARCHAR(20),
description TEXT,
sort_no INT DEFAULT 0,
enabled BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE poi_cache (
  id SERIAL PRIMARY KEY,
  provider VARCHAR(30) NOT NULL,
  provider_poi_id VARCHAR(120),
  name VARCHAR(200) NOT NULL,
  city VARCHAR(80),
  district VARCHAR(80),
  address TEXT,
  lat DOUBLE PRECISION,
  lng DOUBLE PRECISION,
  category VARCHAR(120),
  rating DOUBLE PRECISION,
  cost_level VARCHAR(40),
  is_free BOOLEAN DEFAULT FALSE,
  is_indoor BOOLEAN DEFAULT FALSE,
  cover_image TEXT,
  fetched_at TIMESTAMP,
  expires_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE poi_knowledge (
  id SERIAL PRIMARY KEY,
  poi_id INT REFERENCES poi_cache(id),
  recommend_reason TEXT,
  suitable_people JSONB DEFAULT '[]',
  suitable_weather JSONB DEFAULT '[]',
  suitable_time JSONB DEFAULT '[]',
  best_time VARCHAR(120),
  play_duration VARCHAR(80),
  budget_desc VARCHAR(120),
  avoid_tips TEXT,
  parking_info TEXT,
  toilet_info TEXT,
  child_friendly BOOLEAN,
  elder_friendly BOOLEAN,
  source_type VARCHAR(40) DEFAULT 'manual',
  review_status VARCHAR(30) DEFAULT 'approved',
```

```
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE routes (
  id SERIAL PRIMARY KEY,
  route_no VARCHAR(40),
  title VARCHAR(200) NOT NULL,
  city VARCHAR(80),
  scene_key VARCHAR(40),
  duration_type VARCHAR(40),
  total_distance VARCHAR(80),
  total_time VARCHAR(80),
  budget_desc VARCHAR(120),
  suitable_people JSONB DEFAULT '[]',
  cover_image TEXT,
  summary TEXT,
  tips TEXT,
  source_type VARCHAR(40) DEFAULT 'manual',
  review_status VARCHAR(30) DEFAULT 'approved',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE route_stops (
  id SERIAL PRIMARY KEY,
  route_id INT REFERENCES routes(id),
  poi_id INT REFERENCES poi_cache(id),
  stop_order INT NOT NULL,
  arrive_time VARCHAR(80),
  stay_duration VARCHAR(80),
  transport VARCHAR(80),
  reason TEXT,
  tip TEXT
);
```

13.3 MVP 种子数据建议

- 先选 1 个城市作为种子城市，例如成都。
- 先维护 50-100 个高质量 POI，而不是爬大量低质量数据。
- 每个场景至少准备 8-12 个 POI。
- 先准备 9 条精选路线：每个场景至少 1 条，另加 2 小时、半日、一日通用路线。
- 每个 POI 至少写 1 条推荐理由和 1 条避坑提醒。

13.4 最终验收标准一句话

验收标准

用户打开小程序，不需要先填复杂表单，就能基于当前位置看到“附近现在适合去”的结果；能按场景查、能看精选路线、能问出游助手、能一键导航，并且反馈和联网补充能沉淀进知识库。